# imrestore

# Adaptive and Quasi-Newton First-Order Methods for Image Restoration

Technical note

Abd'gafar Tunde Tiamiyu
School of Mathematics, Jilin University
Changchun 130012, Jilin, China
abdgafartunde@yahoo.com

**Version 0.1.0, October 2026**

---

**Summary**

`imrestore` is an open-source Python package of first-order optimization methods that need no hand-tuned step size, applied to image denoising and deblurring. It implements adaptive gradient descent (AdGD) [1], inertial quasi-Newton forward-backward splitting (QN-FBS) with a rank-one metric [4], and the primal-dual algorithm with linesearch (PDAL) [2] with its variable-metric extension (VarPDAL) [5], together with a Gaussian-blur operator, a total-variation operator, proximal maps, and image-quality metrics. All methods are independent reimplementations from the papers. This note states the problems each method solves, gives the algorithms as implemented with their parameters and stopping rules, reports the verification and restoration results, gives guidance on choosing the parameters, and states the limitations. A step-by-step tutorial is provided separately as `examples/00_getting_started.ipynb`.

## Contents

**1 Introduction** **1**

**2 Operators and proximal maps** **2**

**3 Algorithms** **2**
3.1 Adaptive gradient descent . . . . . 2
3.2 Inertial quasi-Newton forward-backward splitting . . . . . 3
3.3 PDAL and VarPDAL . . . . . 3

**4 Verification** **4**

**5 Restoration results** **5**

**6 Choosing the parameters** **6**

**7 Software reference** **6**

**8 Limitations** **7**

## 1 Introduction

Image restoration recovers an image $x$ from a degraded observation $b = Ax + n$, where $A$ is the identity (denoising) or a blur (deblurring) and $n$ is noise. The standard approach is variational:

minimize a data term plus a regularizer, possibly with constraints such as $x \in [0,1]^N$. The resulting problems take one of three forms, each served by one method:

| problem | method | module in `algorithms` |
|---|---|---|
| $\min_x f(x)$, $f$ smooth | AdGD | `adaptive_gd` |
| $0 \in Az + Bz$, $B$ cocoercive | QN-FBS | `quasi_newton_fbs` |
| $\min_x \max_y \langle Kx, y\rangle + g(x) + h(x) - f^*(y)$ | PDAL, VarPDAL | `var_pdal` |

None of the methods needs the Lipschitz constant of a gradient or the norm of an operator to be known in advance: AdGD estimates the local curvature, and PDAL and VarPDAL find their step sizes by linesearch. QN-FBS and VarPDAL add a rank-one quasi-Newton correction to the metric of the proximal step.

`imrestore` depends only on NumPy, SciPy, and Matplotlib. It is intended for comparing these methods on common problems and as a readable reference implementation.

**Installation.** From the repository root, `pip install -e ".[dev]"` installs the package with the test and notebook dependencies; `uv sync -extra dev` reproduces the exact environment recorded in `uv.lock`.

## 2 Operators and proximal maps

**Gaussian blur.** `GaussianBlur(sigma, kernel_size)` convolves with a normalized Gaussian kernel of size $2\,\mathrm{round}(3\sigma)+1$ by default, as a *circular* convolution computed with the FFT. With periodic boundaries the symmetric kernel makes the operator self-adjoint (`adjoint` returns the same convolution), and because the kernel is nonnegative and sums to one, $\|A\| \le 1$.

**Discrete gradient and total variation.** `TotalVariation(weight)` provides the forward-difference gradient $D : \mathbb{R}^{n_r \times n_c} \to \mathbb{R}^{2 \times n_r \times n_c}$ with zero differences across the last row and column (`forward`), its exact adjoint, which is minus the divergence (`adjoint`), the isotropic TV value $w \sum_{ij} |(Dx)_{ij}|$ (`__call__`), and the projection onto the dual ball $\{p : |p_{ij}| \le w\}$ (`prox_conj`), which is the proximal map of the conjugate of $w\|\cdot\|_{2,1}$. Since TV has no simple proximal map of its own, TV problems are solved in the primal-dual form with $K = D$.

**Proximal maps.** `operators.proximal` provides soft-thresholding (`prox_l1`), shrinkage (`prox_l2_squared`), and projection onto a box (`prox_box`); the first two accept a diagonal metric.

## 3 Algorithms

Every solver returns an `OptimizationResult` with the final iterate `x`, the objective history (when an `objective` callable is given; it is used for diagnostics only), the step-size history, and the number of iterations `n_iter`.

### 3.1 Adaptive gradient descent

`adaptive_gradient_descent(gradient, x0, objective, lambda_0, max_iter, tol)` implements Algorithm 1 of [1] (Algorithm 1). The step size combines growth by at most $\sqrt{1+\theta}$ with a local estimate of the inverse curvature from the last two iterates and gradients. If $f$ is convex with a locally Lipschitz gradient, the iterates converge to a minimizer and a weighted average of them achieves $f - f_* = O(1/k)$; under local strong convexity the convergence is linear [1]. The method stops when $\|\nabla f(x^k)\| \le$ `tol`.

`accelerated_adaptive_gradient_descent` implements the paper's accelerated variant (Algorithm 2), which also estimates the local strong-convexity constant and adds momentum. The paper presents it as a heuristic without a convergence proof.

**Algorithm 1** AdGD (`adaptive_gradient_descent`)

**Require:** $x^0$, $\lambda_0 > 0$ (default $10^{-10}$); $\theta_0 = +\infty$

$x^1 = x^0 - \lambda_0 \nabla f(x^0)$

**for** $k = 1, 2, \ldots$ **do**

$\lambda_k = \min\Big\{ \sqrt{1+\theta_{k-1}}\,\lambda_{k-1},\ \dfrac{\|x^k - x^{k-1}\|}{2\|\nabla f(x^k) - \nabla f(x^{k-1})\|} \Big\}$ ▷ second term dropped if the denominator is zero

$x^{k+1} = x^k - \lambda_k \nabla f(x^k)$; $\quad \theta_k = \lambda_k / \lambda_{k-1}$

**end for**

### 3.2 Inertial quasi-Newton forward-backward splitting

`inertial_quasi_newton_fbs(B, resolvent_A, z0, beta, theta0, inertial_alpha, gamma, objective, max_iter, tol)` solves the monotone inclusion $0 \in Az + Bz$, where $B$ is $\beta$-cocoercive (for example $B = \nabla f$ with $f$ convex and $L$-smooth, $\beta = 1/L$) and $A$ is maximally monotone with a simple resolvent in a scalar metric (for $A = \partial g$, `resolvent_A(v, theta)` $= \mathrm{prox}_{g/\theta}(v)$). It implements Algorithm 1 of [4] with the base metric $M_0 = \theta_0 I$, $\theta_0 > 1/\beta$ (default $1.1/\beta$), and a single rank-one correction per iteration (the paper's 0SR1 update, Section 5):

$$M_k = \theta_0 I \pm \gamma_k\, u_k u_k^\top, \qquad u_k = \frac{r_k}{\sqrt{|\langle r_k, s_k\rangle|}}, \quad r_k = y_k - \theta_0 s_k, \tag{1}$$

with $s_k = z_k - z_{k-1}$, $y_k = Bz_k - Bz_{k-1}$, the sign of $\langle r_k, s_k\rangle$, and no update when that inner product vanishes. For the minus sign, $\gamma_k$ is clipped so that $M_k$ stays positive definite.

Each iteration takes an inertial step $\bar{z}_k = z_k + \alpha(z_k - z_{k-1})$, a forward step in the metric $M_0$, and a backward step in the metric $M_k$. The latter is computed by the paper's resolvent calculus: it reduces to finding the root of a strictly increasing scalar function, found by bracketing (up to 40 expansions) and bisection (60 steps), with two evaluations of the simple resolvent per step; $M_k$ is never formed or inverted. The method stops when $\|z_{k+1} - z_k\| \le$ `tol`.

The paper's convergence theorem requires a summable-type inertial schedule. A constant `inertial_alpha`$> 0$ is a common heuristic that the theorem does not cover; `inertial_alpha = 0` (the default) runs the non-inertial method, which it does.

### 3.3 PDAL and VarPDAL

Both solve the saddle-point problem

$$\min_x \max_y\ \langle Kx, y\rangle + g(x) + h(x) - f^*(y) \tag{2}$$

with $g$ and $f^*$ simple (prox-friendly) and $K$ linear. For TV restoration, $K = D$, $f^*$ is the indicator of the dual ball (`TotalVariation.prox_conj`), $g$ is the indicator of the box $[0,1]^N$ (`prox_box`), and $h(x) = \frac{1}{2}\|Ax - b\|^2$ is the data term.

**PDAL.** `pdal(K, K_adjoint, prox_g, prox_f_conj, x0, y1, tau0, beta, mu, delta, objective, max_iter, tol, max_linesearch)` implements Algorithm 1 of [2] for $h \equiv 0$ (Algorithm 2). It replaces the condition $\sigma\tau\|K\|^2 < 1$ by a backtracking linesearch on the primal step, with the dual step $\beta\tau_k$.

**Algorithm 2** PDAL (`pdal`)

**Require:** $x^0$, $y^1$, $\tau_0 > 0$, $\beta > 0$, $\mu, \delta \in (0,1)$ (defaults 0.7, 0.99); $\theta_0 = 1$

**for** $k = 1, 2, \ldots$ **do**
  $x^k = \operatorname{prox}_{\tau_{k-1} g}\big(x^{k-1} - \tau_{k-1} K^* y^k\big)$
  $\tau_k = \tau_{k-1}\sqrt{1 + \theta_{k-1}}$
  **repeat** ▷ at most `max_linesearch` times
    $\theta_k = \tau_k / \tau_{k-1}$;  $\bar{x}^k = x^k + \theta_k (x^k - x^{k-1})$
    $y^{k+1} = \operatorname{prox}_{\beta\tau_k f^*}\big(y^k + \beta\tau_k K \bar{x}^k\big)$
    **if** $\sqrt{\beta}\,\tau_k \|K^* y^{k+1} - K^* y^k\| \le \delta \|y^{k+1} - y^k\|$ **then break**; **else** $\tau_k \leftarrow \mu\tau_k$
  **until** accepted
  stop if $\|x^k - x^{k-1}\| + \|y^{k+1} - y^k\| \le$ `tol`
**end for**

**VarPDAL.** `var_pdal(K, K_adjoint, prox_g, prox_f_conj, h, grad_h, x1, y0, L_h, sigma0, beta, theta0, gamma, mu, delta, objective, max_iter, tol, max_linesearch)` implements Algorithm 1 of [5] for the full problem (2) with $h$ convex and $\nabla h$ $L_h$-Lipschitz. It linesearches the dual step $\sigma_k$ (with $\tau_k = \beta\sigma_k$) and takes the primal proximal step in a quasi-Newton metric $M_k$:

$$x^{k+1} = \operatorname{prox}^{M_k}_{\tau_k g}\big(x^k - \tau_k M_k^{-1}(K^* \bar{y}^k + \nabla h(x^k))\big),$$

accepting $\sigma_k$ when

$$\tau_k\sigma_k \|Kx^{k+1} - Kx^k\|^2 + 2\tau_k\big(h(x^{k+1}) - h(x^k) - \langle \nabla h(x^k), x^{k+1} - x^k\rangle\big) \le \delta \|x^{k+1} - x^k\|^2_{M_k},$$

and shrinking it by $\mu$ otherwise. The paper's default metric is a multi-memory L-BFGS/L-SR1 metric with a semismooth-Newton resolvent; for consistency and $O(N)$ cost, `imrestore` uses instead the rank-one metric (1) and the scalar resolvent calculus of QN-FBS, with secant pairs from consecutive values of $\nabla h$ and $\theta_0 > L_h$ (default $1.1 L_h$). It stops when the change in the primal and dual iterates is at most `tol`.

## 4 Verification

The package is verified at three levels: the operators, the proximal maps, and the solvers, each against known answers. Figure 1 shows the convergence demos on synthetic problems with closed-form references.

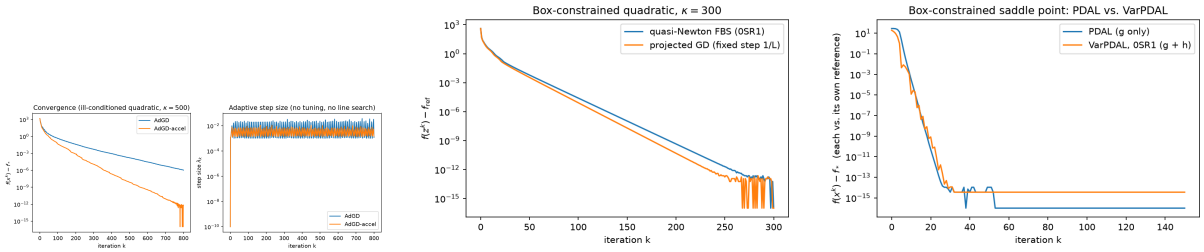



Figure 1: Algorithm demos on synthetic problems. Left: AdGD and its accelerated variant on a quadratic with condition number 500. Centre: QN-FBS against projected gradient descent with step $1/L$ on a box-constrained quadratic with condition number 300. Right: PDAL and VarPDAL on a box-constrained saddle-point problem, each against its own closed-form reference.

- *Operators.* For random inputs, the blur and the discrete gradient pass the dot-product test $\langle Au, v\rangle = \langle u, A^* v\rangle$ to twelve digits; the blur preserves constant images and rejects kernels larger than the image.

- *Proximal maps.* `prox_l1` matches the closed form and satisfies the optimality condition; `prox_l2_squared` and `prox_box` match their closed forms; `prox_conj` projects onto the dual ball.
- *Solvers.* Each solver converges to the known minimizer of a test problem: AdGD and its accelerated variant on quadratics (the variant needs fewer iterations on an ill-conditioned one), QN-FBS on an unconstrained and a box-constrained quadratic (with and without inertia), PDAL on a saddle-point problem with a closed-form solution, and VarPDAL on the same problem with an added smooth term. Invalid parameters ($\theta_0 \le 1/\beta$, $\theta_0 \le L_h$) are rejected.
- *Convergence demos.* `scripts/demo_*.py` compare each method with a baseline on these problems (Figure 1).

The test suite has 31 tests. Continuous integration runs it on Python 3.10 to 3.13, runs every script from outside the repository, and executes both notebooks; the benchmark notebook reproduces its stored outputs exactly.

## 5 Restoration results

All results use the $96 \times 96$ Shepp–Logan phantom with values in $[0, 1]$.

**TV restoration with VarPDAL.** `scripts/restoration_demo.py` runs two experiments (Figures 2 and 3):

- denoising with noise $\sigma = 0.08$ and TV weight $w = 0.1$, 200 iterations: PSNR 23.6 to 25.8 dB, SSIM 0.40 to 0.62;
- deblurring a Gaussian blur with $\sigma = 1$ and noise $\sigma = 0.01$, with $w = 10^{-3}$ and 3000 iterations: PSNR 19.7 to 29.3 dB, SSIM 0.76 to 0.93.

Both weights favour visual quality over PSNR. For denoising, PSNR peaks at $w = 0.05$ (27.8 dB) and SSIM at $w = 0.07$ (0.631); $w = 0.1$ gives flatter regions at 25.8 dB and SSIM 0.623. For deblurring, a sweep with the same settings gives PSNR 30.8 dB at $w = 3 \times 10^{-4}$ to $5 \times 10^{-4}$ and 29.3 dB at $10^{-3}$, while SSIM peaks at $10^{-3}$ (0.927).

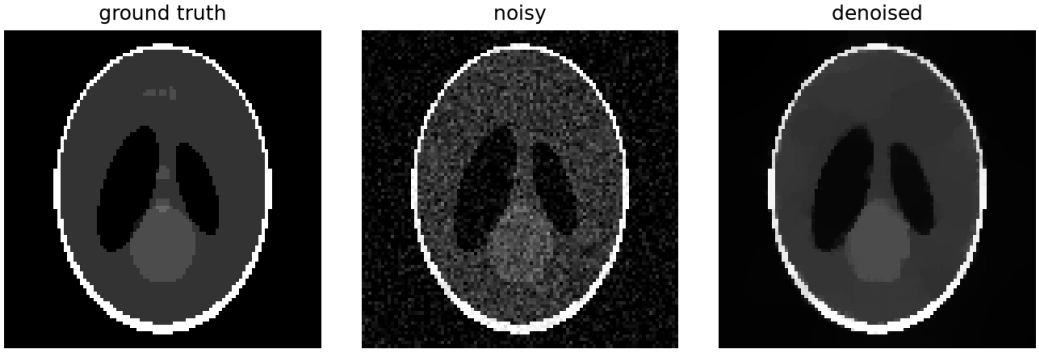



Figure 2: TV denoising with VarPDAL ($\sigma = 0.08$, $w = 0.1$).

**The three methods on denoising.** The benchmark notebook (`examples/01_restoration_benchmark.ipynb`) denoises with noise $\sigma = 0.12$ (PSNR 20.19 dB, SSIM 0.320). AdGD and QN-FBS minimize a smooth Tikhonov objective with a gradient penalty (QN-FBS adds the box constraint) and reach 21.34 dB (SSIM 0.350) in 38 and 30 iterations; VarPDAL with TV ($w = 0.08$, 200 iterations) reaches 24.40 dB (SSIM 0.562).

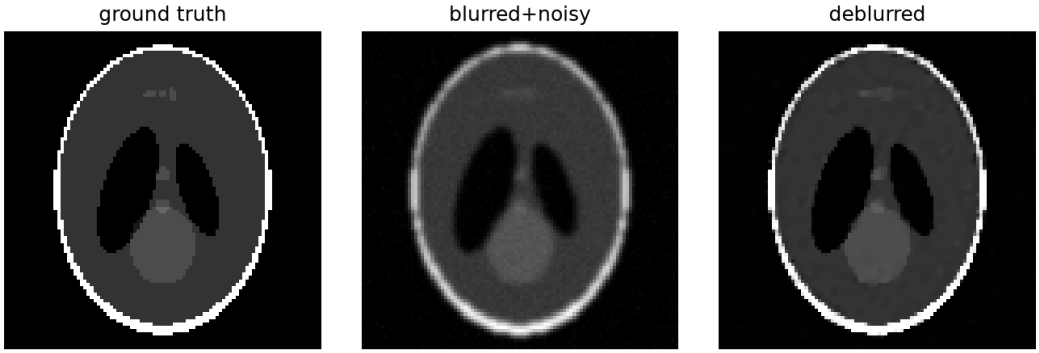



Figure 3: TV deblurring with VarPDAL (blur $\sigma = 1$, noise 0.01, $w = 10^{-3}$).

**Step sizes.** In the tutorial's deblurring problem with a smooth objective ($L \leq 1.08$), AdGD reaches machine precision in about 200 iterations, while gradient descent with the fixed step $1/L$ is still near $10^{-10}$ after 300. Its median step is 1.76, about twice $1/L$, and its largest is 17.6.

**Choosing the TV weight.** In the tutorial (deblurring without clipping the data), the discrepancy principle [3], which picks the weight whose residual $\|Ax_w - b\|$ matches the noise norm $\sigma\sqrt{N}$, selects $w = 10^{-3}$ (PSNR 29.3 dB, SSIM 0.963, the highest SSIM in the sweep); the PSNR optimum is at $w = 3 \times 10^{-4}$ (30.9 dB). VarPDAL with 3000 iterations takes about 5 s at this size on an Apple M-series laptop.

## 6 Choosing the parameters

**AdGD.** Leave `lambda_0` at its default; the step grows to the local curvature scale within a few iterations. Set `tol` on the gradient norm, and pass `objective` only to record the history.

**QN-FBS.** Pass the cocoercivity constant `beta` $= 1/L$ of $B$; for $f(x) = \frac{1}{2}\|Ax - b\|^2 + \frac{\mu}{2}\|Dx\|^2$ with the blur and gradient above, $L \leq \|A\|^2 + 8\mu \leq 1 + 8\mu$. Keep `inertial_alpha = 0` if you need the convergence guarantee.

**PDAL and VarPDAL.** The initial step (`tau0` or `sigma0`) only needs to be of the right order: the linesearch adjusts it. The ratio `beta` between primal and dual steps matters more; `beta = 1` works for TV restoration with images in $[0, 1]$. For VarPDAL pass $L_h = \|A\|^2$, which is at most 1 for the normalized blur. Small TV weights make the problem close to unregularized deblurring and need thousands of iterations; check that the result no longer changes when `max_iter` is doubled.

**Regularization weight.** Use the discrepancy principle when the noise level is known, and compare the images as well as PSNR and SSIM, which can disagree for TV.

## 7 Software reference

Table 1 lists the public functions; every solver returns an `OptimizationResult` (Section 3).

**Reproducing the results.** The figures in this note are in `figures/` and are regenerated by the scripts in `scripts/`, which run from any directory and accept `-outdir`.

Table 1: Public functions (module prefixes relative to `imrestore`).

| function | purpose |
|---|---|
| `algorithms.adaptive_gd`: `adaptive_gradient_descent`, `accelerated_adaptive_gradient_descent` | AdGD and its accelerated variant |
| `algorithms.quasi_newton_fbs`: `inertial_quasi_newton_fbs` | QN-FBS with the 0SR1 metric |
| `algorithms.var_pdal`: `pdal`, `var_pdal` | PDAL and VarPDAL |
| `operators.blur.GaussianBlur(sigma, kernel_size)` | periodic Gaussian blur |
| `operators.tv.TotalVariation(weight)` | gradient, adjoint, TV value, dual projection |
| `operators.proximal.prox_l1`, `prox_l2_squared`, `prox_box` | proximal maps |
| `metrics.image_quality.psnr`, `ssim` | PSNR and SSIM (data range 1 by default) |
| `utils.io.load_phantom(name, size)`, `add_gaussian_noise` | test images (`"shepp-logan"`, `"circles"`) and noise |
| `utils.visualization.plot_convergence`, `plot_restoration_grid` | plotting |

# 8 Limitations

- *Metrics.* QN-FBS and VarPDAL use a rank-one quasi-Newton metric only; the multi-memory L-BFGS/L-SR1 metrics of the papers are not implemented.
- *Guarantees.* The accelerated AdGD variant and QN-FBS with a constant inertial parameter are heuristics without the papers' guarantees; the accelerated variants of PDAL are not implemented.
- *Models.* Grayscale images, Gaussian blur with periodic boundaries, and Gaussian noise; no Poisson data terms, no spatially varying blur.
- *Implementation.* Pure NumPy on the CPU.

# References


[1] Y. Malitsky and K. Mishchenko. Adaptive gradient descent without descent. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, PMLR 119, 2020. arXiv:1910.09529.

[2] Y. Malitsky and T. Pock. A first-order primal-dual algorithm with linesearch. *SIAM Journal on Optimization*, 28(1):411–432, 2018.

[3] V.A. Morozov. On the solution of functional equations by the method of regularization. *Soviet Mathematics Doklady*, 7:414–417, 1966.

[4] S. Wang, J. Fadili, and P. Ochs. Inertial quasi-Newton methods for monotone inclusion: efficient resolvent calculus and primal-dual methods. arXiv:2209.14019, 2022 (v3, 2024). See also: Quasi-Newton methods for monotone inclusions: efficient resolvent calculus and primal-dual algorithms, *SIAM Journal on Imaging Sciences*, 18(1):308–344, 2025.

[5] S. Wang, J. Fadili, and P. Ochs. A quasi-Newton primal-dual algorithm with line search. arXiv:2405.06824, 2024. See also the conference paper with the same title in *Scale Space and Variational Methods in Computer Vision*, Lecture Notes in Computer Science, pages 444–456, Springer, 2023.